

# Lava API Android App — Implementation Plan (Sub-project 1)

**For agentic workers:** REQUIRED SUB-SKILL: Use `superpowers:subagent-driven-development` (recommended) or `superpowers:executing-plans` to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

**Goal:** Ship a standalone `digital.vasic.lava.api` Android app that boots the full Lava API in-process (cross-compiled Go via gomobile, SQLite-backed), exposes it on the LAN with mDNS, and is controllable (start/stop/restart/status) from a landing screen + foreground notification — without breaking the existing Postgres-backed Go API.

**Architecture:** Additive storage-backend selector in `lava-api-go` (postgres default untouched + new pure-Go SQLite impl) → thin `internal/mobile/gomobile` surface → `gomobile bind .aar` → `:core:apiengine` Kotlin wrapper → `:api-app` Compose app with `ApiEngineService` (foreground), `NsdManager` advertisement, notification.

**Tech Stack:** Go 1.26, `modernc.org/sqlite` (pure-Go, CGo-free), `gomobile bind`, Android NDK, Kotlin/Jetpack Compose, Hilt, Orbit, `NsdManager`, `JUnit4`, `orbit-test`, Compose UI test / Espresso.

---

## Pre-flight (do once, before Phase A; surface blockers honestly per §6.L/§11.4.6)

☐

**PF1: Verify Go toolchain.** Run `go version` (expect  $\geq$  `go1.26`). Record output.

☐

**PF2: Verify gomobile availability.** Run `gomobile version` or which `gomobile`. If absent, note as a Phase-B blocker (install path: `go install golang.org/x/mobile/cmd/gomobile@latest && gomobile init`); Phase A does NOT need `gomobile`.

☐

**PF3: Verify Android NDK.** Check `$ANDROID_HOME/ndk` or `local.properties`. Needed for `gomobile bind` (Phase B) only.

☐

**PF4: Confirm `modernc.org/sqlite` resolvable.** Run `go list -m modernc.org/sqlite` in `lava-api-go/`. If not a direct dep, `go get modernc.org/sqlite` is Task A2-step.

☐

Record all PF outputs in the kickoff commit body. Any missing tool → name it as a blocker, do not bluff downstream phases.

---

## Phase A — lava-api-go additive SQLite backend (no Android, fully testable locally)

**Context every Phase-A task must load first:** Read lava-api-go/internal/config/config.go, lava-api-go/internal/cache/cache.go, lava-api-go/internal/server/ (composition root), and one existing lava-api-go/internal/\*/integration\_test.go to learn the REAL signatures (config.Load, server constructor, cache interface). Do not guess signatures — adapt the code below to what you find. Run Go tests with `cd lava-api-go && GOMAXPROCS=2 nice -n 19 go test ./...` (§6.T.2 resource limits).

### Task A1: Storage backend config selector (additive, default = postgres)

#### Files:

- Modify: lava-api-go/internal/config/config.go
- Test: lava-api-go/internal/config/config\_storage\_test.go (new)
- Modify: .env.example (add placeholders)



**Step 1: Read** internal/config/config.go fully; note the Config struct fields and Load() validation block (the LAVA\_API\_PG\_URL is required error at ~line 145).



**Step 2: Write failing test** config\_storage\_test.go:

```
func TestLoad_DefaultBackendIsPostgresAndStillRequiresPgUrl(t
    *testing.T) {
    t.Setenv("LAVA_API_PG_URL", "")
    _, err := config.Load() // default backend = postgres
    if err == nil { t.Fatalf("expected PG_URL-required error on
        default backend") }
}

func TestLoad_SqliteBackendRequiresPathNotPgUrl(t *testing.T) {
    t.Setenv("LAVA_API_STORAGE_BACKEND", "sqlite")
    t.Setenv("LAVA_API_PG_URL", "")
    t.Setenv("LAVA_API_SQLITE_PATH", "/tmp/lava-test.db")
    cfg, err := config.Load()
    if err != nil { t.Fatalf("sqlite backend must not require
        PG_URL: %v", err) }
    if cfg.StorageBackend != "sqlite" || cfg.SQLitePath != "/tmp/
        lava-test.db" {
        t.Fatalf("unexpected cfg: %v", cfg)
    }
}
```

```
func TestLoad_SqliteBackendRequiresPath(t *testing.T) {
    t.Setenv("LAVA_API_STORAGE_BACKEND", "sqlite")
    t.Setenv("LAVA_API_SQLITE_PATH", "")
    if _, err := config.Load(); err == nil { t.Fatal("sqlite
        backend must require SQLITE_PATH") }
}
```



**Step 3: Run** go test ./internal/config/ -run TestLoad\_ -v → expect FAIL (fields/branch missing).



**Step 4: Implement** in config.go: add StorageBackend string (default "postgres" when env empty) and SQLitePath string; change the validation so PGUrl is required iff StorageBackend == "postgres", and SQLitePath is required iff StorageBackend == "sqlite"; reject unknown backends.



**Step 5: Run** the test → expect PASS. Then run the **full existing config suite** go test ./internal/config/ → expect ALL PASS (regression guard: existing tests set PG\_URL and must be unaffected).



**Step 6:** Add LAVA\_API\_STORAGE\_BACKEND=postgres and LAVA\_API\_SQLITE\_PATH=/data/lava-api.db placeholders to .env.example.



**Step 7: Commit** with Bluff-Audit stamp (mutation: flip the iff postgres guard to unconditional; observe TestLoad\_SqliteBackendRequiresPathNot PGUrl fails; revert).

## Task A2: Define the storage interface boundary

### Files:

- Read: lava-api-go/internal/cache/cache.go (what the postgres cache provides today)
- Create: lava-api-go/internal/storage/storage.go (interface + types)
- Test: lava-api-go/internal/storage/storage\_test.go



**Step 1: Read** internal/cache/cache.go and every caller of it (grep -rn "cache\." internal/) to enumerate the exact methods the server depends on (e.g. Get/Set/TTL, auth/session persistence). Write the enumerated method list into the commit body — this is the interface contract.



**Step 2: Write** storage.go declaring type Storage interface { ... } with exactly those methods, plus type Backend string constants. No implementation.



**Step 3: Write** storage\_test.go with a conformance helper func RunStorageConformance(t \*testing.T, newStore func() Storage) asserting behavioral invariants (set→get round-trip, TTL expiry, missing-key behavior) that BOTH backends must satisfy (this is the parity harness).



**Step 4: Run** `go test ./internal/storage/` → expect FAIL (no impls yet)  
— acceptable; the harness is consumed by A3/A4.



**Step 5: Commit.**

### Task A3: Postgres impl = wrap existing behavior (no behavior change)

#### Files:

- Create: `lava-api-go/internal/storage/postgres.go`
- Test: `lava-api-go/internal/storage/postgres_test.go` (gated  
— `-Pintegration/-tags=integration`, real Postgres in podman)



**Step 1:** Implement `postgresStorage` delegating to the existing `internal/cache/postgres` adapter — zero behavior change.



**Step 2:** `postgres_test.go` runs `RunStorageConformance` against a real podman Postgres (reuse existing integration-test harness; same build tag).



**Step 3: Run** integration test → PASS. Run full existing suite → PASS (regression).



**Step 4: Commit** (Bluff-Audit: break the get to return wrong value; conformance fails; revert).

### Task A4: SQLite impl (pure-Go modernc.org/sqlite)

#### Files:

- Create: `lava-api-go/internal/storage/sqlite.go`
- Create: `lava-api-go/migrations/sqlite/0001_init.sql` (SQLite-dialect schema mirroring the postgres migrations)
- Test: `lava-api-go/internal/storage/sqlite_test.go` (no external service — uses a temp-file DB)



**Step 1:** `go get modernc.org/sqlite` if not present; import `"modernc.org/sqlite"` and `database/sql` with driver name `"sqlite"`.



**Step 2: Write** `sqlite_test.go` that builds a `sqliteStorage` on `t.TempDir()` + `"/t.db"` and runs `RunStorageConformance` — expect FAIL first.



**Step 3: Implement** `sqlite.go`: open DB, run `migrations/sqlite/*.sql`, implement every `Storage` method using SQL (TTL via an `expires_at` column + lazy expiry on read).



**Step 4: Run** `go test ./internal/storage/ -run Sqlite -v` → PASS.



**Step 5: Commit** (Bluff-Audit: make TTL expiry a no-op; conformance TTL case fails; revert).

## Task A5: Wire backend selection into the composition root

### Files:

- Modify: lava-api-go/internal/server/ composition root (where cache is constructed today) + lava-api-go/cmd/lava-api-go/main.go
- Test: lava-api-go/internal/storage/parity\_test.go



**Step 1: Read** the server constructor + main.go to find where the postgres cache is built and injected.



**Step 2:** Add a storage.New(cfg) (Storage, error) factory selecting impl by cfg.StorageBackend; inject the resulting Storage where the cache was used. Postgres path = identical to today.



**Step 3: Write** parity\_test.go: spin the real HTTP handlers twice (once sqlite temp-file, once — gated — postgres), issue identical requests (health + a representative search/browse with a stubbed upstream), assert **status + JSON body field-equality**.



**Step 4: Run** parity test (sqlite leg always; postgres leg under integration tag) → PASS.



**Step 5: Run the ENTIRE lava-api-go suite** go test ./... → ALL PASS (final Phase-A regression gate).



**Step 6: Commit** (Bluff-Audit: point the factory at the wrong impl; parity field-equality fails; revert). **Phase A done — the host API now runs on SQLite via config, Postgres untouched.**

---

## Phase B — internal/mobile native surface + c-shared .so build

**DECISION 2026-06-02 (operator):** gomobile bind is BLOCKED by lava-api-go's 16 relative replace ../submodules/\* directives (gomobile's overlay-module generator writes a 0-byte go.mod; proven on NDK 21.4 + 25.1). Replaced with **go build -buildmode=c-shared** (honors replace directives) + a hand-written JNI bridge. Still in-process, still cross-compiled Go. The internal/mobile Go package (B1, commit b03150eb) is done + tested; the gomobile build script (B2, d1022a1f) is superseded by the c-shared build (B2'). Requires NDK clang as CGO cross-compiler (CGO\_ENABLED=1, CC=NDK clang per ABI).

**Scope fix:** the embed must serve the FULL production router (the same handlers cmd/lava-api-go builds — search/browse/topic/auth), bound to 0.0.0.0 (designed network exposure) with a self-signed TLS cert generated at first boot, SQLite-backed — not just /health+/ready on loopback. Bind addr validated via net.ParseIP (security-review hardening) while explicitly allowing non-loopback.

## Task B1: internal/mobile package

**Files:** Create lava-api-go/internal/mobile/mobile.go; Test lava-api-go/internal/mobile/mobile\_test.go.



**Step 1: Write mobile\_test.go:** Start(configJSON) on a free loopback port (sqlite temp db), then a **real http.Get to the server's health URL asserting 200 + real JSON body**, then Stop(), then assert the port is closed. Status() returns JSON containing "backend":"sqlite".



**Step 2: Run** → FAIL.



**Step 3: Implement mobile.go:** package-level server handle guarded by a mutex; Start(configJSON string) error parses JSON → config.Config, builds storage + server, listens on a goroutine, returns once accepting (or error); Stop() error graceful shutdown; Status() string returns JSON. Exported surface uses only string/error (clean gomobile/JNI types).



**Step 4: Run** → PASS.



**Step 5: Commit** (Bluff-Audit: make Start return before listening; the real http.Get fails; revert).

## Task B2: gomobile bind build script + reproducible artifact

**Files:** Create lava-api-go/scripts/build-aar.sh; Create docs/scripts/build-aar.sh.md (§11.4.18); output lava-api-go/build/lavaapi.aar.



**Step 1: Script runs**  
gomobile bind -target=android -androidapi 28 -o build/lavaapi.aar ./internal/mobile with GOMAXPROCS=2 nice -n19 and prints the output .aar sha256.



**Step 2: Run** the script → expect a real .aar; record sha256. If gomobile/NDK missing → BLOCKER, surface to operator.



**Step 3: Decide artifact handling** (prefer: built locally + cached, not committed; document in the .md). Add build/ to .gitignore if needed.



**Step 4: Commit** the script + doc (not the binary).

---

## Phase C — :core:apiengine Kotlin wrapper module

Context: Read core/notifications/ + an existing core/\* module's build.gradle.kts + a convention plugin in buildSrc/ to mirror module setup.

### Task C1: Module skeleton + ApiEngine interface + behaviorally-equivalent fake

**Files:** Create core/apiengine/build.gradle.kts, core/apiengine/src/main/kotlin/lava/apiengine/ApiEngine.kt, .../ApiStatus.kt, core/testing fake FakeApiEngine; modify settings.gradle.kts (include(":core:apiengine")); Test core/apiengine/src/test/kotlin/lava/apiengine/FakeApiEngineTest.kt.



**Step 1:** Add module to settings; apply lava.android.library (or lava.kotl in.library if no Android dep needed beyond the aar — the aar needs Android, so android.library).



**Step 2:** Define interface ApiEngine { suspend fun start(config: ApiConfig): Result<ApiStatus>; suspend fun stop(): Result<Unit>; fun status(): ApiStatus } and data class ApiStatus(state, bindAddr, port, requestCount, backend, version).



**Step 3:** Write FakeApiEngineTest asserting the fake enforces start-before-running and propagates a configured error (Third Law behavioral equivalence).



**Step 4:** Implement FakeApiEngine to pass.



**Step 5:** Run ./gradlew :core:apiengine:test → PASS. Commit.

### Task C2: Real GomobileApiEngine delegating to the .aar

**Files:** Create .../GomobileApiEngine.kt; add the .aar as a module dependency (flatDir or libs/).



**Step 1:** Wire the lavaapi.aar into core/apiengine deps.



**Step 2:** Implement GomobileApiEngine calling the generated Mobile.start/stop/status, mapping JSON status → ApiStatus, dispatching on Dispatchers.IO.



**Step 3:** Compile (./gradlew :core:apiengine:assembleDebug). Real behavior is asserted by the instrumented Challenge in Phase E (in-process server can't fully run in a JVM unit test). Commit.

---

## Phase D — :api-app module (Compose app, Service, notification, mDNS)

Context: Read app/build.gradle.kts (appId/suffix/signing/version pattern), app/src/main/AndroidManifest.xml, core/notifications/NotificationServiceImpl.kt, and core/data/.../LocalNetworkDiscoveryService\* (NsdManager usage) to mirror conventions.

### Task D1: :api-app module + manifest + signing + appId

**Files:** Create api-app/build.gradle.kts, api-app/src/main/AndroidManifest.xml, api-app/src/main/kotlin/lava/api/app/ApiApplication.kt; modify settings.gradle.kts.



**Step 1:** applicationId = "digital.vasic.lava.api", debug applicationIdSuffix = ".dev", independent versionCode=1/versionName="0.1.0", **same signing config as :app** (reuse the keystore config block), depend on :core:apiengine, :core:designsystem, :core:notifications. Permissions: INTERNET, ACCESS\_NETWORK\_STATE, ACCESS\_WIFI\_STATE, CHANGE\_WIFI\_MULTICAST\_STATE, FOREGROUND\_SERVICE, FOREGROUND\_SERVICE\_DATA\_SYNC, POST\_NOTIFICATIONS, WAKE\_LOCK.



**Step 2:** Hilt @HiltAndroidApp Application. Compile ./gradlew :api-app:assembleDebug → PASS. **Commit.**

### Task D2: ApiEngineService foreground service (start/stop/restart lifecycle + locks + mDNS)

**Files:** Create api-app/src/main/kotlin/lava/api/app/service/ApiEngineService.kt; Test api-app/src/test/kotlin/lava/api/app/service/ApiEngineServiceLogicTest.kt (logic extracted to a testable controller).



**Step 1:** Extract lifecycle/state logic into ApiEngineController(engine: ApiEngine, advertiser: MdnsAdvertiser, ...) exposing StateFlow<ApiControlState> so it's unit-testable with FakeApiEngine + fake advertiser.



**Step 2: Write ApiEngineServiceLogicTest** (real controller + FakeApiEngine): start → state Running with url/ips; stop → Stopped; restart → Stopped→Starting→Running; engine error → Error state. Primary assertions on emitted state.



**Step 3:** Implement controller to pass; Service holds Multicast/Wifi/Wake locks while Running, releases on Stop.



**Step 4: Run** ./gradlew :api-app:testDebugUnitTest → PASS. **Commit** (Bluff-Audit: make restart skip the Starting state; assertion fails; revert).

### Task D3: mDNS advertiser (NsdManager, engine=go platform=android storage=sqlite)

**Files:** Create `.../service/MdnsAdvertiser.kt` (interface) + `NsdMdnsAdvertiser.kt`; Test `.../MdnsAdvertiserTxtTest.kt`.



**Step 1: Write** `MdnsAdvertiserTxtTest` asserting the built `NsdServiceInfo` uses service type `_lava-api._tcp` (release) / `_lava-api-dev._tcp` (debug) and TXT attrs `engine=go`, `platform=android`, `storage=sqlite`, `version=...` (Build the TXT map via a pure function so it's unit-testable.)



**Step 2:** Implement; register/unregister on start/stop.



**Step 3: Run** test → PASS. **Commit.**

### Task D4: Landing screen (MVI/Orbit) + notification

**Files:** Create `.../control/ApiControlViewModel.kt`, `ApiControlState/Action/SideEffect.kt`, `.../control/ApiControlScreen.kt`, `.../notification/ApiNotification.kt`; Test `.../control/ApiControlViewModelTest.kt` (orbit-test, real controller, FakeApiEngine).



**Step 1: Write** `ApiControlViewModelTest`: Start action → state transitions to Running with reachable URL + IP list + request count; Stop → Stopped; Restart → through Starting; error → Error message. No `_` in any rendered label (assert formatting helper).



**Step 2:** Implement `ViewModel` + state machine to pass.



**Step 3:** Compose `ApiControlScreen`: status indicator, Start/Stop/Restart buttons (enabled per state), reachable `https://<ip>:<port>`, all LAN IPs, request counter, plain-language explanation of what running means + how other devices connect.



**Step 4:** Foreground notification (channel via `core:notifications`): title/body with URL + count, **Stop/Restart actions**, tap → landing screen.



**Step 5: Run** `./gradlew :api-app:testDebugUnitTest` → PASS. **Commit** (Bluff-Audit: make Stop leave state Running; assertion fails; revert).

---

## Phase E — Instrumented Challenge tests (real device/emulator, real evidence — §6.J/§6.Z/§6.AE)

Runner: host-direct + HVF per §6.L 67th cycle (darwin/arm64). Requires a provisioned AVD; if none, execution is honestly BLOCKED, not faked. Each Challenge carries a FALSIFIABILITY REHEARSAL KDoc block.

## Task E1: Challenge01\_ApiAppColdStartSurvives

**Files:** Create `api-app/src/androidTest/kotlin/lava/api/app/challenges/Challenge01ApiAppColdStartTest.kt`.

☐

**Step 1:** `pm clear digital.vasic.lava.api.dev`; launch; assert the landing screen renders (status visible) without crash.

☐

**Step 2:** Falsifiability KDoc: throw in `ApiApplication.onCreate` → cold-start fails. **Commit.**

## Task E2: Challenge02\_ApiAppBootAndServesRealJson (load-bearing)

**Files:** Create `.../Challenge02ApiAppBootAndServeTest.kt`.

☐

**Step 1:** Launch → tap **Start** → wait until UI shows **Running** with a URL.

☐

**Step 2:** From the test process, issue a **real HTTPS request to the on-device server's health + a search/browse endpoint**, assert **200 + real JSON body fields** (primary assertion on the wire).

☐

**Step 3:** Falsifiability KDoc: make `mobile.Start` bind but serve 500 on health → the JSON assertion fails with a clear message.

☐

**Step 4:** **Commit.**

## Task E3: Challenge03\_StopRestart + Challenge04\_NotificationActions

**Files:** Create the two Challenge files.

☐

Stop → UI Stopped + subsequent HTTPS request refused; Restart → back to Running + serving. Notification Stop/Restart actions drive the same transitions. Falsifiability KDoc each. **Commit.**

## Task E4: Execute the Challenge matrix + capture evidence

☐

**Step 1:** Build `:api-app:assembleDebug` + `:api-app:assembleDebugAndroidTest`.

☐

**Step 2:** Run via the host-direct+HVF runner against a provisioned AVD (min API 28/30/34/latest per §6.AE.2 where the AVD set exists). Capture connected `DebugAndroidTest BUILD SUCCESSFUL` verbatim.

☐

**Step 3:** Write `.lava-ci-evidence/<...>/api-app-real-device-verification.{md,json}` with per-(Challenge×AVD) rows (§6.I.4 Group-B fields). If the AVD set is incomplete → record honestly which API levels ran and which are BLOCKED.



**Step 4: Commit** evidence.

---

## Self-review notes

- **Spec coverage:** §2 spec→Phase A; §3.1 gomobile→Phase B; §3.1 :core:apiengine→Phase C; §3.2/3.3/3.4/5→Phase D; §4 discovery→Task D3; §6 testing→Phases A tests + E. Roadmap §8 (sub-projects 2–4) intentionally out of this plan (own specs).
- **Type consistency:** `ApiEngine.start/stop/status` + `ApiStatus(state,bindAddr,port,requestCount,backend,version)` used consistently C1→C2→D2→D4. Storage interface defined A2, implemented A3/A4, selected A5, consumed B1.
- **No placeholders:** implementation steps that depend on unread signatures explicitly begin with a "read the named file" step (no-guessing §11.4.6) rather than fabricated signatures.
- **Honest blockers:** PF2/PF3 (gomobile/NDK) gate Phase B; AVD gates Phase E; neither is faked.